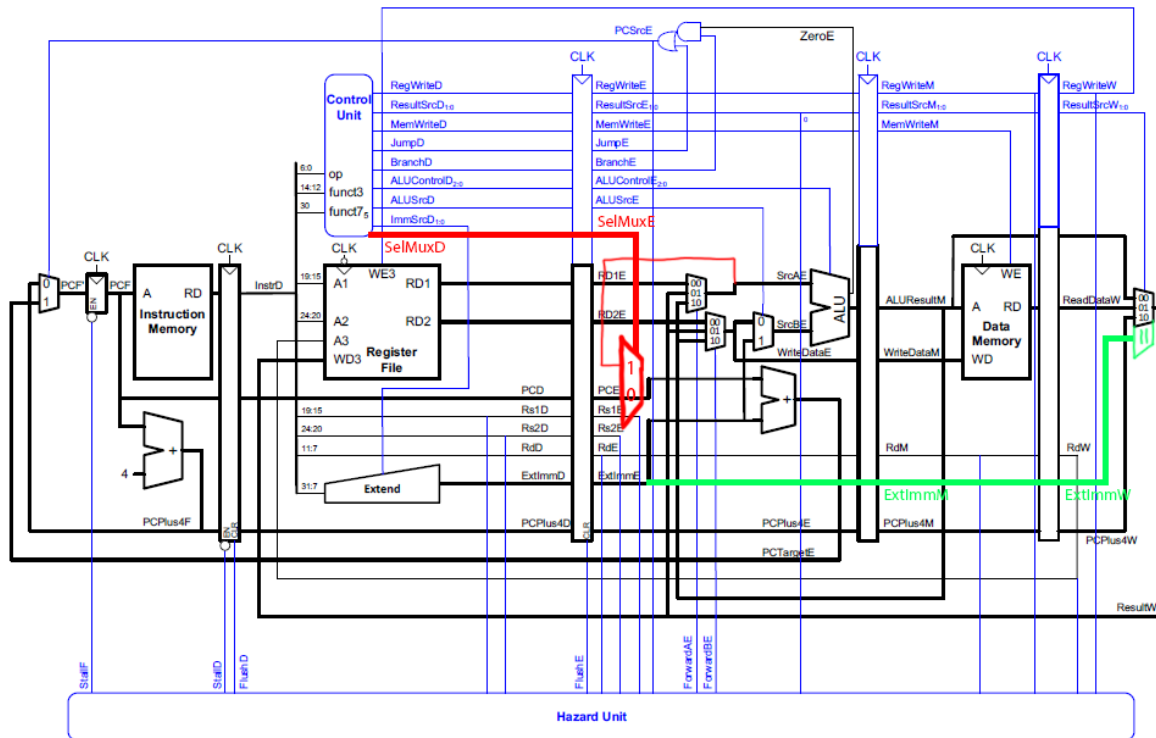


سیستم های دیجیتال 2 | تمرین کامپیوتری 4

متن شیا سی 810101453 | آرن ریاحی 810101433

Datapath:

RISC-V Pipelined Processor with Hazard Unit



Controller:

Instruction Type	Instructions	Configurations
R-type	add, sub, and, or, slt	- RegWrite = 1, ALUOp = 10, ALUSrc = 0, MemWrite = 0, ResultSrc = 00, Branch = 0, Jump = 0 ALUFunc: add: 000, sub: 001, and: 010, or: 011, slt: 100
I-type	lw	- RegWrite = 1, ImmSrc = 000, ALUSrc = 1, MemWrite = 0, ResultSrc = 01 - ALUOp = 00, Branch = 0, Jump = 0 - ALUFunc = 000
	addi, xori, ori, slti	- RegWrite = 1, ImmSrc = 000, ALUSrc = 1, MemWrite = 0, ResultSrc = 00 - ALUOp = 11, Branch = 0, Jump = 0 - ALUFunc: - addi: 000 - xori: 100 - ori: 011 - slti: 100
	jalr	- RegWrite = 1, ImmSrc = 000, ALUSrc = 1, MemWrite = 0, ResultSrc = 10 - ALUOp = 00, SelMux = 1, Branch = 0, Jump = 1 - ALUFunc = 000 - Operation: - jalr: $ra = PC + 4$, $PC = rs1 + adr$
S-type	sw	- RegWrite = 0, ImmSrc = 001, ALUSrc = 1, MemWrite = 1, ResultSrc = 00 - ALUOp = 00, Branch = 0, Jump = 0 - ALUFunc = 000
J-type	jal	- RegWrite = 1, ImmSrc = 010, MemWrite = 0, ResultSrc = 10 - ALUOp = 00, SelMux = 1 - Operation: - jal: $ra = PC + 4$, $PC = PC + adr$
B-type	beq, bne	- RegWrite = 0, ImmSrc = 001, ALUSrc = 0, MemWrite = 0, ALUOp = 01, ResultSrc = 00 - SelMux = 0, Branch = 1, Jump = 0
U-type	lui	- RegWrite = 1, ImmSrc = 100, MemWrite = 0, ResultSrc = 11 - Branch = 0, Jump = 0

Instructions and Memory:

```
addi x8,x8 , 0x000; -> 00000000000001000000010000010011
addi x5, x0, 1; -> 000000000000100000000001010010011
sw x5, 0(x8); -> 0000000010101000010000000100011
addi x5, x0, 2; -> 000000000010000000000001010010011
sw x5, 4(x8); -> 00000000010101000010001000100011
addi x5, x0, 42; -> 000000101010000000000001010010011
sw x5, 8(x8); -> 00000000010101000010010000100011
addi x5, x0, 32; -> 000000100000000000000001010010011
sw x5, 12(x8); -> 00000000010101000010011000100011
addi x5, x0, 12; -> 000000001100000000000001010010011
sw x5, 16(x8); -> 00000000010101000010100000100011
sw x5, 20(x8); -> 00000000010101000010101000100011
addi x5, x0, -56; -> 111111001000000000000001010010011
sw x5, 24(x8); -> 00000000010101000010110000100011
addi x5, x0, 7; -> 000000000111000000000001010010011
sw x5, 28(x8); -> 00000000010101000010111000100011
addi x5, x0, -45; -> 111111010011000000000001010010011
sw x5, 32(x8); -> 00000010010101000010000000100011
addi x5, x0, 8; -> 000000001000000000000001010010011
sw x5, 36(x8); -> 00000010010101000010001000100011
addi x6, x0, 1; -> 000000000001000000000001100010011
lw x9, 0(x8); -> 00000000000001000001001001000011
addi x18, x18, 0; -> 00000000000010010000010010010011
addi x5, x5, 40; -> 00000101000001010000001010010011
loop:
beq x18, x5, end_loop; -> 0000001001011001000000001100011
add x19, x18, x8; -> 0000000100100100000000100110011
lw x20, 0(x19); -> 000000000000100110100101000011
slt x21, x9, x20; -> 0000000101000100101001010110011
beq x21, x6, end_if; -> 0000000001101010100000001100011
addi x9, x20, 0; -> 00000000000010100000010010010011
end_if:
addi x18, x18, 4; -> 00000000010010010000010010010011
jal loop; -> 111111001011111111111111101111
end_loop:
```

Result:

